

Лабораторная работа I. Подготовка данных

1 Как выполнять лабораторные работы

Лабораторные работы рекомендуется выполнять по порядку, поскольку каждая работа так или иначе зависит от предыдущих работ.

1.1 GitHub

Для выполнения лабораторных работ необходимо создать репозиторий на GitHub. Репозиторий может быть публичным, может быть приватным, выбор за вами.

Добавьте преподавателя в репозиторий на правах коллаборатора. Для этого в репозитории перейдите по **Settings** → **Collaborators** → **Add people**, в появившемся окне найдите аккаунт преподавателя и пошлите ему приглашение присоединиться к репозиторию.

Для каждой лабораторной работы создавайте отдельную git-ветку. Для сдачи лабораторной работы сделайте pull-request (он же merge-request) этой ветки и выберите преподавателя для код-ревью.

1.2 Структура проекта

Весь основной код вы должны реализовывать в **.py модулях**.

По ходу выполнения лабораторной работы, вам потребуется выводить промежуточные результаты. Для этого вы можете либо использовать jupyter-блокноты, импортируя там ваши .py модули, либо написать CLI-приложение с выводом всех промежуточных результатов в консоль.

Если вы хотите продемонстрировать какие-то результаты, которые нет возможности показать в jupyter-блокноте или CLI-приложении, можете добавить их в **README.md-файл**. Материалы в этом файле вы также можете использовать на защите.

Рекомендуемая структура проекта, от которой можно отталкиваться:

```
README.md
.gitignore
requirements.txt      - фиксация окружения проекта
configs/              - конфигурационные файлы
data/                 - данные
notebooks (или cli)/  - .ipynb блокноты или .py CLI-скрипты
```

src/	
data/	- .py файлы для обработки данных
tokenization/	- .py файлы для токенизации
models/	- .py файлы для моделей
...	
utils.py	- утилитарные функции
checkpoints	- чекпоинты функции

На GitHub не надо грузить данные и чекпоинты (за некоторыми исключениями).

1.3 Как набрать максимальный балл за работу

За каждое задание предусмотрены свои баллы, всего 20 баллов на каждую лабораторную работу (а на самом деле можно получить до 25 баллов).

Дедлайн работы — 2 недели со дня ее публикации, в 23:59 по московскому времени. То есть, если работа опубликована 5 мая в 14:44, то ее дедлайн — 19 мая в 23:59.

Работа считается сданной в момент пуша коммита с сообщением "lab<номер ЛР> final". Преподавателем будет проверяться версия кода из этого коммита. В случае возникновения критической ошибки во время работы, задание будет считаться невыполненным (поэтому рекомендуется писать тесты для своей работы, чтобы ничего не сломать в последний момент).

Оставшиеся задания можно будет загрузить и после просроченного дедлайна, но со штрафом 0.65. То есть, если задание стоит 5 первичных баллов, то в случае просроченного дедлайна за него будет выставлено 3.25 балла.

Если работа была сдана за неделю до дедлайна, то это считается ранней сдачей работы. За раннюю сдачу баллы за работу умножаются на коэффициент 1.25. Досылать задания после ранней сдачи нельзя, иначе работа перестает считаться таковой. Можно только вносить исправления для код-ревью.

Перед защитой работы необходимо внести все правки из код-ревью.

Чтобы баллы были проставлены в БАРС, работу необходимо защитить. На защите задаются вопросы по коду и по теории. В случае неуспеха на защите, штрафа за повторную попытку не будет, можно попробовать еще раз :)

1.4 Что делать, если нет необходимого железа для обучения моделей

Если у вас нет доступа к необходимому оборудованию (например, к видеокарте NVIDIA), выполняйте код в Colab и Kaggle (подключив необходимое окружение). Чтобы работать с вашим кодом, сделайте git clone репозитория в местную файловую систему.

Ноутбук с выводами ячеек можно скачать с Colab или Kaggle и загрузить на GitHub, выводы ячеек в этом случае сохранятся.

2 Рекомендации к выполнению лабораторной работы

Для выполнения работы рекомендуется использовать язык Python и фреймворк `lightning` (а именно `LightningDataModule` для подготовки данных). Остальные библиотеки в этой работе выбираются на ваше усмотрение.

Главное — фиксируйте окружение, в котором выполняете работу, в файле `requirements.txt`.

При большом желании можно реализовать BPE-токенизатор на C/C++ и подключить его к проекту. Для этого можно использовать `pybind11`.

<https://pybind11.readthedocs.io/en/stable/index.html>

Рекомендуется также разделять "сырые" датасеты (`raw`) и обработанные (`processed`). Опционально: можете подключить систему контроля версий для данных, например — DVC <https://dvc.org/>.

3 Задание

3.1 Скачать датасет (1 балл)

Для обучения языковой модели необходимы текстовые данные. Текстовых данных много в интернете. Давайте скачаем интернет (вернее, его часть).

Common Crawl — это архив веб-страниц, собранных автоматическим обходом интернета. Он составлял костяк датасета, на котором обучалась GPT-3. Данные Common Crawl хранятся не как текст, а как архив результатов обхода, поэтому содержат множество метаданных (которые для обучения языковой модели нам не понадобятся).

Common Crawl — слишком большой датасет, поэтому скачаем только один WARC-файл.

В этом задании вам необходимо:

- Скачать часть датасета Common Crawl — WARC-файл. При желании можете скачать несколько WARC-файлов, если не боитесь занять слишком много места на диске.
- Конвертировать его в текстовый формат.

3.2 Очистить данные (3 балла)

Вспомним bias-variance decomposition. Ошибка модели раскладывается на несколько компонент: bias^2 , variance и σ^2 . Первые две компоненты мы можем уменьшить во время обучения модели, но шумовую компоненту σ^2 таким образом не устранить, она заложена в самих данных.

Для одного и того же текста следующий токен не определен однозначно. Например, это может быть следствием допущенной в тексте ошибки (тот

самый шум σ^2 в данных). Однако даже если мы избавимся от всех ошибок в данных, текст все равно не будет определен однозначно, поскольку некоторые слова могут трактоваться по-разному.

Это — энтропия языка и для естественного человеческого языка она ненулевая. Однако, чем более грязный датасет используется для обучения, тем выше неопределенность следующего токена.

Для функции потерь кросс-энтропии энтропия языка — теоретический минимум. Допустим, мы его достигли в ходе обучения модели. Если модель изначально обучается на шумных данных, то и её генерации будут содержать больше мусора и ошибок.

Поэтому чем чище данные на которых мы обучаем, тем лучшего результата мы можем достичь в ходе обучения модели. **В этом задании вам необходимо:**

- В первую очередь, необходимо удалить HTML-разметку и заголовки (header'ы).
- Далее удалите неизвестные языки и символы. Неизвестные языки — это те, на которых мы точно не будем обучать модель.
- Дополнительно можете добавить фильтрацию объектов по ключевым словам, чтобы отсеивать токсичные данные (это необязательный пункт).
- Нормализуйте пробелы и Unicode-символы.
- Удалите пустые строки.
- Слишком длинные объекты разбейте на объекты поменьше. Заранее оцените свои вычислительные ресурсы для оценки длины последовательности. Рекомендуется обучать на модель на последовательностях длиной 512-1024 токенов (это примерно 350-700 слов в каждом объекте для английского языка).

3.3 Улучшаем качество данных (6 баллов)

Мы выполнили первичную очистку датасета. Однако данные все еще довольно грязные: в тексте могут встретиться дублирующиеся абзацы, куски с повторяющимися многократно фразами, или наоборот — бессвязный бред.

Будем разбирать проблемы по порядку. Начнем с дубликатов. Очевидно, что многие веб-страницы копируют друг друга полностью или частично. То, во сколько раз корпус текста больше за счёт повторов, называется **дублирующей плотностью данных**:

$$\rho_{\text{dub}} = \frac{|D|}{|\text{unique}(D)|}$$

Дублирование данных можно трактовать как добавление дополнительных эпох при обучении модели, если предположить, что дубликаты распределены равномерно. Высокая дублирующая плотность не является особо

страшной проблемой при обучении малых моделей, но при обучении LLM она крайне негативно сказывается на законах масштабирования.

Перейдем к следующей проблеме: если участок текста представлен случайным набором символом или токенов, без какой-либо структуры, то участок текста сильно повышает энтропию данных. Средняя энтропия датасета называется **информационной плотностью данных**:

$$\rho_{\text{info}}(D) = \frac{H(D)}{|D|} = -\frac{1}{|D|} \sum_{i=1}^{|D|} \log p(D_i | D_{<i})$$

Для вычисления $H(D)$, необходимо оценить распределение p . Для этого мы можем использовать предобученную модель GPT-2. Скачайте модель с Hugging Face. Для инициализации модели используйте библиотеку `transformers`. Пропустите через модель объекты вашего датасета.

В этом задании вам необходимо:

- Для каждого объекта датасета вычислить энтропию $H(D)$ с помощью модели GPT-2 и информационную плотность.
- Оценить информационную плотность всего датасета (подсказка: используйте взвешенное среднее) и вывести её.
- Удалить объекты-дубликаты (если есть) и объекты с крайне низкой и крайне высокой энтропией языка (границы оставлены на ваше усмотрение).

3.4 Токенизация текста (6 баллов)

Текст очищен, теперь необходимо конвертировать его в формат, понятный для языковой модели. **Реализуйте токенизацию:**

- Реализуйте токенизацию по символам. Выведите размер словаря и размер последовательности случайного объекта из датасета.
- Реализуйте токенизацию по словам. Выведите размер словаря и размер последовательности случайного объекта из датасета. Если у вас не хватает памяти для хранения всего словаря — обучите токенизатор лишь на части объектов.
- Реализуйте BPE-токенизацию

3.5 Датасет для обучения модели (4 балла)

Скачайте датасет `wikitext` (тут ссылка на `hugging face`). Он достаточно малый по сравнению с `Common Crawl`.

Выполните все предыдущие задания, но теперь с `wikitext`. Код у вас уже написан, просто вызовите его снова, но для нового датасета. Исключение — BPE токенизатор, используйте тот, что обучен на `Common Crawl`.

После получения всех результатов **реализуйте механизм packed batching**.

При формировании батчей данных необходимо чтобы все объекты были одной длины. Простой способ — добавить <PAD>-токены для каждой последовательности, чтобы увеличить их до размера самой длинной последовательности.

Чтобы эти <PAD>-токены не влияли на вычисление механизма внимания, для каждого объекта создадим маску. Если на этом индексе находится <PAD>-токен, то значение маски равно 0, иначе — 1.

Допустим, мы хотим получить батчи объектов длиной 512 токенов. Однако, у нас осталось всего два объекта: один длиной 300 токенов, другой — 210. Добавляя <PAD>-токены, мы увеличиваем длины последовательностей в 2 раза, что крайне расточительно с точки зрения расхода ресурсов.

Packed batching — это способ объединить несколько коротких объектов в один. В случае с нашим примером, мы можем склеить два объекта в один и получить последовательность длиной в 510 токенов, к которой мы добавим два <PAD>-токена.

Чтобы не смешивать позиционное кодирование и внимание между двумя объектами, обновим маску. Для <PAD>-токенов значение маски остается равным 0, для первого объекта — 1, для второго — 2 (и так далее, если мы объединили большее число объектов).